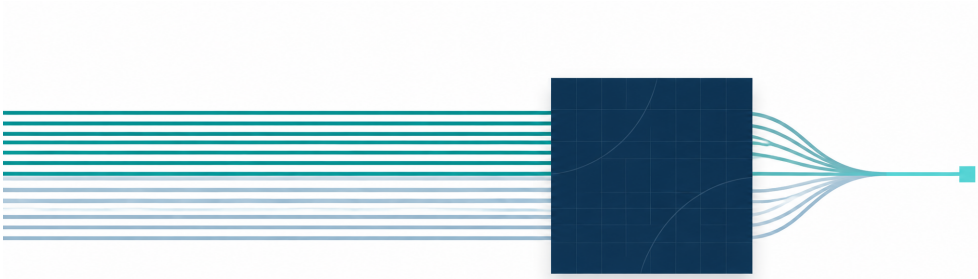
香山昆明湖 V2

自定义向量点积指令

第二阶段设计与基础测试文档

技术报告



工程组合： 0f ＋ firstdot V2 ＋原 C8

团队名称： 待团队确认

成员： 待团队确认

学校： 待团队确认

材料日期： 2026 年 11 月 2 日

文档修订： 2026 年 11 月 5 日

目录

第二阶段：向量点积设计与功能验证 1

2.1 指令语义与微架构设计 1

2.1.1 点积定义、位宽与结果布局 1

2.1.2 指令编码与合法配置 2

2.1.3 架构参考伪代码 5

2.1.4 P/R 两槽数据通路 5

2.2 RTL 功能单元扩展与父级接入 8

2.2.1 译码、微操作与配置依赖 8

2.2.2 逐拍状态转移与握手 9

2.2.3 VFEX4 派发与共享资源 12

2.2.4 时钟门控、唤醒与写回 14

2.3 基础功能验证 15

2.3.1 正数点积测试 16

2.3.2 负数点积测试 16

2.3.3 边界点积测试 16

2.3.4 完整定向向量与精确预期 17

2.3.5 数据布局、别名与异常测试 18

2.3.6 随机输入与完整结果读回 19

2.4 协议与系统验证 20

2.4.1 反压期间的结果与身份保持 20

2.4.2 ROB 取消与复位优先级 21

2.4.3 DIV 切换、取消清理与关钟 22

2.4.4 写回身份与全核结果检查 23

2.4.5 控制场景的组合与回归覆盖 24

2.5 设计演进、参数扩展与取舍 24

2.5.1 吞吐与实现成本 24

2.5.2 本阶段小结与接口衔接 25

2.6 复现资料与接口索引 **25**
2.6.1 汇编封装与编码冲突记录 25
2.6.2 源码接入位置与职责 25
2.6.3 参考模型的构建与运行身份 27
2.6.4 事件记录与检查不变量 27
2.6.5 原 O2 分阶段判据与负控 28
2.6.6 RVV 对照的功能等价条件 29
2.6.7 待确认规格与变更记录 29
2.6.8 技术资料索引 30

第二阶段：向量点积设计与功能验证

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

本设计在香山昆明湖 V2 现有向量执行单元中接入自定义点积，采用乘积槽 P 和结果槽 R 保存两笔在途请求。下文依次说明指令语义、RTL 实现、正数、负数与边界测试，以及取消和共享资源控制；完成取消关钟修复的最终工程版本记为 0f。

2.1　指令语义与微架构设计

点积将多组乘法结果归约为一个和。本设计在昆明湖 V2 已有向量执行路径中接入 vdot，复用译码、向量寄存器、调度、写回和精确提交机制。数据通路完成 8 项有符号乘加，控制通路维护请求身份、反压及取消。优化围绕独立点积吞吐、结果寄存器开销和与 DIV 共享父级资源的切换行为展开。

2.1.1　点积定义、位宽与结果布局

从指令执行前的源寄存器状态读取：

```
byte_a[i] = vs1_before[8*i+7 : 8*i]
byte_b[i] = vs2_before[8*i+7 : 8*i]      i = 0…7
signed8(x) = x           if x < 128
           x - 256     otherwise
```

源 [127:64] 不参与运算。小端裸机加载将数组第 0 个字节映射到源 [7:0]，后续字节依次对应更高的 8 位字段。向量十六进制打印通常从高位开始，检查时按位段还原 lane 编号，与数组下标逐项对应。

单项乘积范围为 [-16256, 16384]，因此八项精确和为：

```
S_min = 8 × (-128) × 127    = -130048
S_max = 8 × (-128) × (-128) =  131072
```

有符号 18 位的正上限为 131071，八项精确和的正端点达到 131072，因此归约至少使用 19 位有符号表示。架构结果使用 32 位补码，覆盖全部合法输入的精确范围。正端点由八项最大正乘积共同形成，是归约末级位宽的确定性边界。

算术实现由 8 个 signed 8×8 乘积和归约树组成。乘积使用 16 位，二项、四项、八项归并分别保留 17、18、19 位，再扩展为 32 位结果。各级先按符号扩展操作数再相加，保留对应进位。采用 32 位中间和也满足同一数学定义；具体结构改变时，完整结果比较继续使用精确整数参考值。

```
127                          32 31                 0
+-------------------------------------+-------------------------+
|          全部为 0                   |    S 的 32 位补码       |
+-------------------------------------+-------------------------+
```

例如 S=-8 时，完整目的值为 0x00000000_00000000_00000000_FFFFFFF8。补码位于低 32 位结果字段，高 96 位为零。该字段按完整寄存器写回约定解释，结果读回以全部 16 字节为检查对象。

vl=8 限定本指令的合法输入配置，输出则完整覆盖 128 位目的寄存器。通用 tail/mask 合并路径在接入时按这一规则配置或绕开，确保目的高 96 位取零，旧目的值不参与结果。四种 vta/vma 组合使用相同的输出定义。

2.1.2 指令编码与合法配置

本项目选择 32 位 custom-0 主 opcode 0x0b，funct 字段由内部 v0.1 合同定义。标准 opcode 表提供自定义扩展空间，编码分配在团队使用范围内固定。

表 2-1 指令编码与合法配置

位段	字段	v0.1 取值
[31:26]	funct6	000000
[25]	vm	1；本版只支持无掩码
[24:20]	vs2	第二个编码源寄存器编号，0～31
[19:15]	vs1	第一个编码源寄存器编号，0～31
[14:12]	funct3	000
[11:7]	vd	目的向量寄存器编号，0～31
[6:0]	opcode	0001011（0x0b）

汇编记法沿用向量常用顺序 vdot.vv vd, vs2, vs1。数学公式中 vs1/vs2 的名称与位段一致；点积交换律会掩盖源颠倒，所以编码测试还必须检查字段提取本身。

```
MATCH_VDOT = 0x0200000b
MASK_VDOT  = 0xfe00707f
识别条件： (instruction & MASK_VDOT) == MATCH_VDOT
编码公式： MATCH_VDOT | (vs2 << 20) | (vs1 << 15) | (vd << 7)
BitPat：  b0000001??????????000?????0001011
```

vdot.vv v10, v8, v9 编码为 0x0284850b，小端字节序列为 0b 85 84 02。三个寄存器编号均指向量寄存器。编码测试按位段提取 vs2、vs1 和 vd，并核对译码产生的向量源、目的类型。

vm=0 或固定 funct 字段不符时不匹配本版 vdot。精确 mask/match 只接受已定义模式，其余编码由其他已分配扩展或默认非法指令路径处理，邻近模式分别进入译码测试。

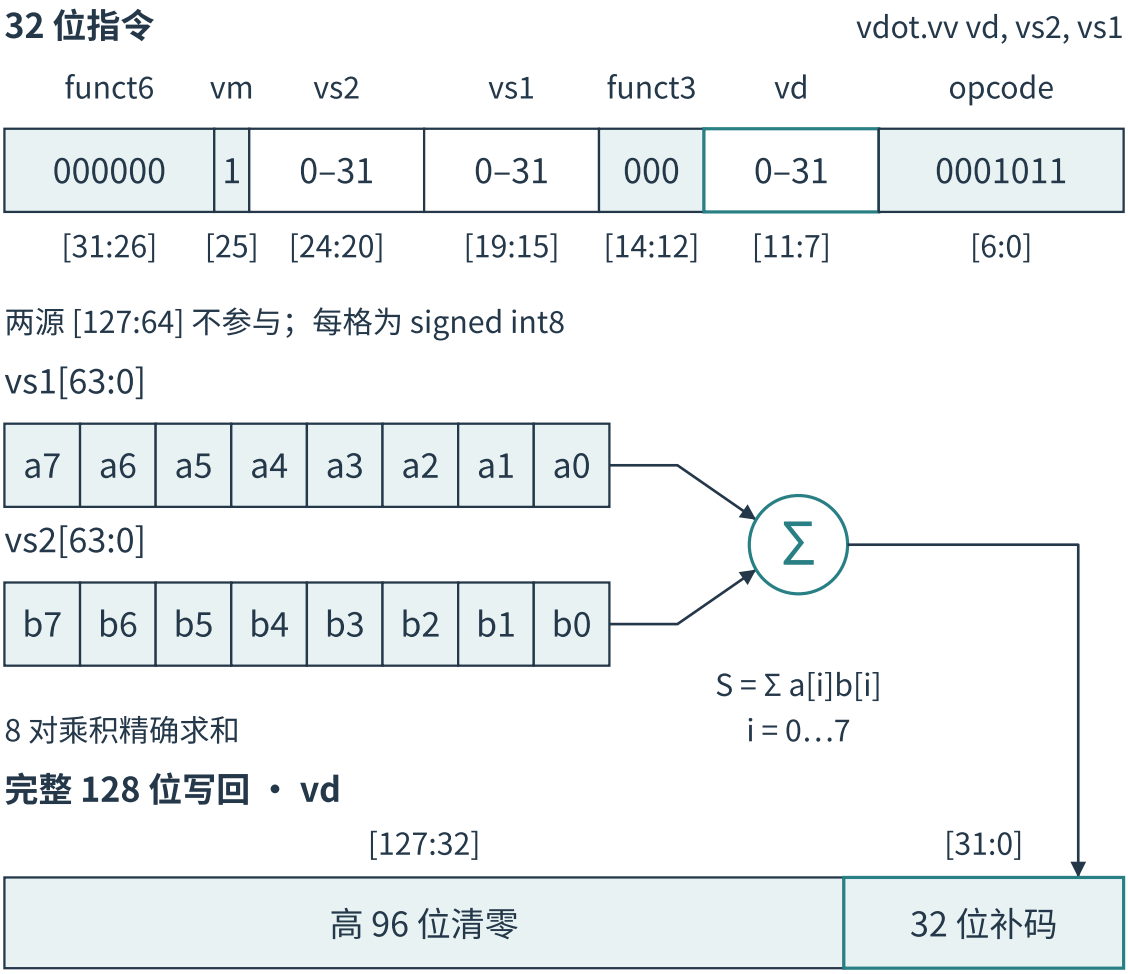


图 2-1　vdot 指令字段与完整结果布局

低 8 个有符号字节逐项相乘并求和；结果低 32 位保存补码，高 96 位清零。内部合同 vdot-s8x8-r32-v0.1；Q01–Q07 待官方澄清；先读源、后覆盖 vd（允许别名）。

表 2-2　指令编码与合法配置

条件	指令行为与测试条件
VLEN	固定为 128；RTL 构建与 NEMU 配置使用相同常量，其他 VLEN 采用独立版本约定
向量状态	运行环境支持并启用 V，mstatus.VS 不为 Off；沿用既有特权态检查
vtype	vill=0、vsew=000（e8）、vlmul=000（m1）；域外配置进入非法指令路径
vl	等于 8；vl=0、1、7、9、16 等值属于域外配置，其中 vl=0 按非法配置处理
vstart	等于 0；非零值进入非法指令路径

表 2-2 指令编码与合法配置（续）

条件	指令行为与测试条件
vm	等于 1；vm=0 相邻编码由默认译码及其他已分配扩展规则处理
vta/vma	tu/mu、tu/ma、ta/mu、ta/ma 均合法；输出仍精确遵循“点积定义、位宽与结果布局”小节

基础测试使用 vsetivli zero, 8, e8, m1, tu, mu 建立配置，读取实际 vl/vtype 并核对 vstart=0，再执行已匹配的 vdot 机器字。配置准备与自定义指令译码是两个步骤，测试分别保留其状态与指令身份。

v0.1 内部合同采用固定的 custom 指令合法域，并定义完整 128 位结果写回。标准向量状态、策略位和重启机制用于现有体系接入；vdot 的低 8 项读取、域外异常和目的格式由本合同明确规定。相关背景见RISC-V Vector 1.0 文档。

vd/vs1/vs2 均为 v0…v31，每个操作数只占一个寄存器，无偶数寄存器对齐要求。v0 在本版 vm=1 下可作为普通源或目的，不是零寄存器。

源值取自指令执行前的寄存器快照，再写回目的。NEMU 先缓存所需源数据，RTL 通过重命名维护源版本与目的映射；vd 与任一源相同、两个源相同以及三者相同都遵循这个顺序。旧 vd 不参与数学累加，通用旧目的端口也不改变该结果。

成功执行时：

- 只更新一个向量目的寄存器的完整 128 位；其余向量寄存器、标量寄存器、浮点寄存器、内存不变。
- vl、vtype、vlenb、vxrm、vxsat、vcsr 保留原值；合法初态下 vstart 仍为 0，原有 vxsat=1 同样保留。
- 沿用现有向量状态置脏机制；不产生浮点异常标志，不访问数据存储器。
- 正常进入下一条指令（本编码长度 4 字节），退休与计数行为沿用处理器正常指令机制。

编码匹配而配置不合法时，合同规定精确非法指令异常，异常码为 2，陷阱 PC 指向该指令。目的寄存器、运算相关向量 CSR 和内存保持指令执行前状态。陷阱入口对特权 CSR 的正常更新、tval 填写和异常优先级沿用现有机制。测试优先构造单一可达非法条件；vill 与 vl 等联动条件按“数据布局、别名与异常测试”小节成组记录。

VLEN 一致性在构建或加载配置时检查。跨 VLEN 扩展需要独立规定读取范围、结果布局与异常条件，并以新的合同版本组织实现和测试。

redirect/flush 按 ROB 年龄取消乱序请求，被取消事务不产生已提交的目的更新。指令采用完整操作的精确提交，沿用现有中断与取消机制，不提供中间 lane 的架构恢复状态。全核验证以异常、取消和正常恢复用例核对这些行为。

2.1.3 架构参考伪代码

以下伪代码定义架构效果。它使用精确整数求和、先读源后写目的，并把合法性检查与数据运算分开；实际握手和时钟行为见“逐拍状态转移与握手”和“时钟门控、唤醒与写回”。

```
def execute_vdot(inst, state):
  # 必须先完成编码、向量状态和配置检查；失败则走非法指令路径。
  require_vdot_encoding_and_supported_state(inst, state)

  src1 = read_full_vector_before_write(inst.vs1)  # 128-bit snapshot
  src2 = read_full_vector_before_write(inst.vs2)  # 128-bit snapshot
  total = 0                              # 数学整数，不模拟 RTL 截断
  for i in range(8):
    a = (src1 >> (8 * i)) & 0xff
    b = (src2 >> (8 * i)) & 0xff
    a = a if a < 128 else a - 256
    b = b if b < 128 else b - 256
    total += a * b

  result128 = total & 0xffffffff              # 仅编码低 32 位补码
  write_full_vector(inst.vd, result128)       # 高 96 位因此为零
  # vl/vtype/vxrm/vxsat/vcsr 不变；vstart 仍为 0；向量状态按写回置脏。
```

独立数学模型负责输入数据与结果计算；完整指令模型在其外层加入编码匹配、向量状态、配置检查和精确异常处理。模型的输入检查用于识别测试数据是否合法，处理器异常则按指令模型和已有陷阱机制产生。

2.1.4 P/R 两槽数据通路

两槽实现采用 FuConfig.piped=false 与 UncertainLatency() 框架接口。piped 字段表达框架对确定时延的要求，P/R 则表达 FU 内部缓冲状态；两槽弹性推进与不定时延响应在同一接口下配合。反压、取消和父级资源互斥共同决定实际完成时机。

VDot 使用独立 FuncUnit 维护两笔请求的状态，输入可复用 VecFuncUnitAlias，P、R 分别保存各自的事务包。原单份 RegEnable/DataHoldBypass 身份结构按单在途设计，两槽路径因此独立管理有效位、结果和控制。原 O2 合同以完整 128 位结果定义逻辑接口；当前 H1/0f 将结果状态物理保存为 32 位，输出拼接高 96 位零。

表 2-3 P/R 两槽数据通路

槽位	有效状态与数据	必须一同保存的身份
P：乘积槽	valid、8 个 16 位有符号乘积、输入合法性	接受时的 ROB、pdest、vecWen/v0Wen、需要输出的完整 vpu 控制及 perfDebugInfo；字段清单按 FuncUnitOutput 逐项核对
R：结果槽	valid、32 位结果状态、异常向量；输出端拼接为完整 128 位	从 P 转移的同一包身份，与该结果状态一同保存

被接受的非法配置请求携带异常状态进入槽，并以对应身份终结一次。v0 与普通向量目的分别使用匹配的写使能和 pdest。输出数据、异常、vpu 控制及 perf 信息统一取自 R 槽，在反压期间保持完整包稳定。

父级通过 P、R 真实 valid 或精确 occupancy/empty 归约获知 VDot 占用，派发和门控据此决策。观察 seq 用于关联日志，保持在硬件功能之外。若验证端另有计数副本，逐拍断言其与真实槽状态一致，使功能状态、父级选择和检查器账本可相互核对。

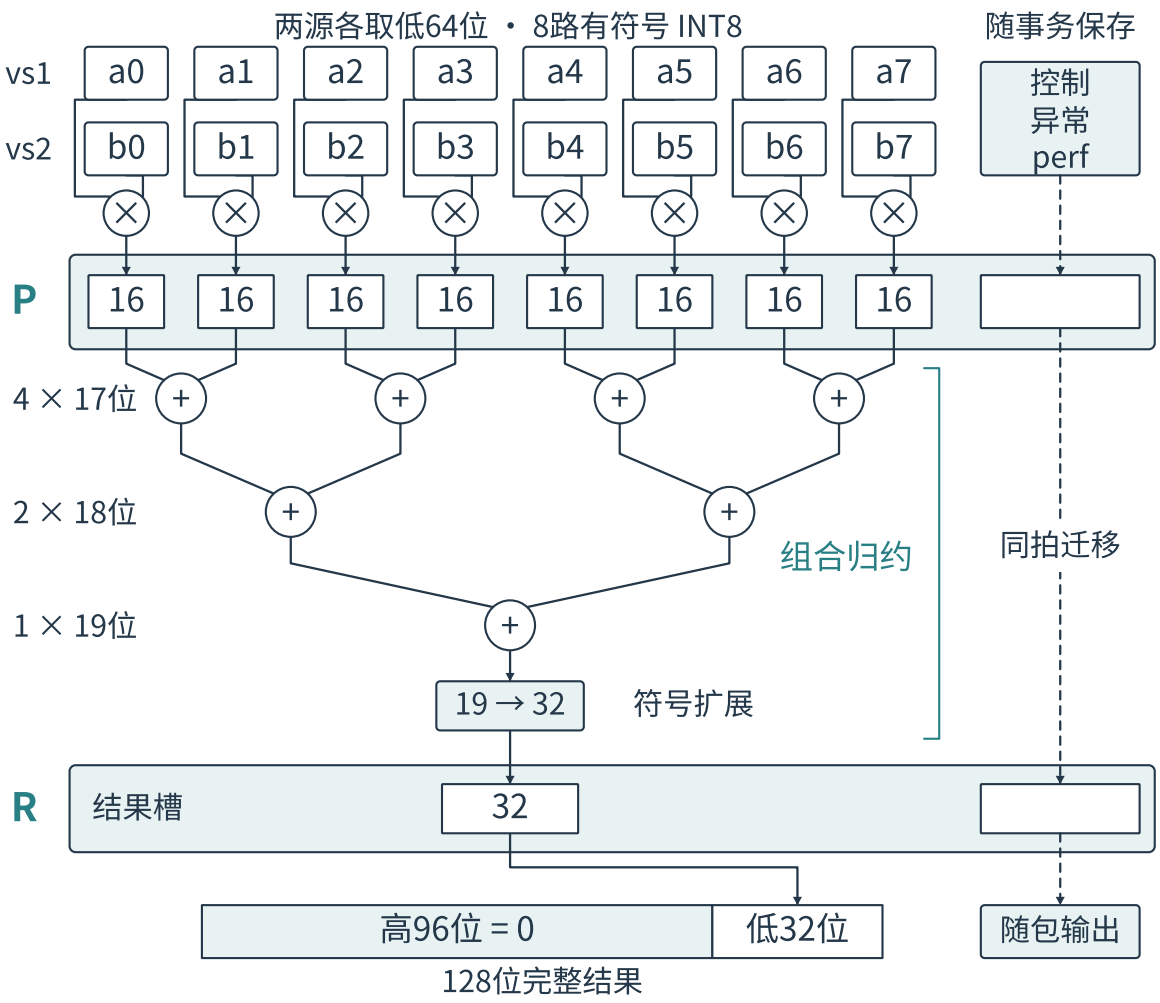


图 2-2 0f 点积算术与 P/R 两槽数据通路

8 路乘积经 P 槽、组合归约和 R 槽输出，事务身份随数据传递。持续 ready、允许接收且无取消时，局部 FU 接口最早 2 拍响应，连续接收间隔为 1 拍。

```scala
class VDotProducts extends RawModule {
  val io = IO(new Bundle {
    val vs1 = Input(UInt(128.W))
    val vs2 = Input(UInt(128.W))
    val products = Output(Vec(8, SInt(16.W)))
  })

  for (lane <- 0 until 8) {
    val a = io.vs1(8 * lane + 7, 8 * lane).asSInt
    val b = io.vs2(8 * lane + 7, 8 * lane).asSInt
    io.products(lane) := a * b
  }
}

/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
```

(a) VDotProducts 乘积计算

```scala
/** Balanced, widening signed reduction of eight 16-bit products.
  * Only the low 32 bits receive the sign-extended sum; bits 127:32 are zero.
  * No mask, tail, old destination, rounding, or saturation is involved.
  */
class VDotReduce extends RawModule {
  val io = IO(new Bundle {
    val products = Input(Vec(8, SInt(16.W)))
    val result = Output(UInt(128.W))
  })

  private val sums17 = Wire(Vec(4, SInt(17.W)))
  private val sums18 = Wire(Vec(2, SInt(18.W)))
  private val sum19 = Wire(SInt(19.W))

  for (pair <- 0 until 4) {
    sums17(pair) := io.products(2 * pair) +& io.products(2 * pair + 1)
  }
  for (pair <- 0 until 2) {
    sums18(pair) := sums17(2 * pair) +& sums17(2 * pair + 1)
  }
  sum19 := sums18(0) +& sums18(1)

  private val result32 = Wire(SInt(32.W))
  result32 := sum19
  io.result := Cat(0.U(96.W), result32.asUInt)
}

/** Combinational arithmetic top for independent verification.
  * The future functional-unit wrapper instantiates the same two submodules
  * separately, with registers between them; this top has no cycle protocol.
  */
```

(b) VDotReduce 归约求和

图 2-3 VDot 乘法与归约代码

VDotProducts 生成 8 路有符号乘积，VDotReduce 完成归约求和。

2.2 RTL 功能单元扩展与父级接入

2.2.1 译码、微操作与配置依赖

译码按照精确 mask/match 识别 vdot，将 vs1、vs2 和 vd 解释为向量寄存器字段，并建立与本条指令对应的配置依赖。执行采用单个微操作、单次完整向量写回。编码测试同时检查字段位置和寄存器类别，使机器字、源读取与目的写回保持一致。

实际 VL 通过重命名后的依赖传入执行端，SEW、LMUL、vill 等控制信息随微操作传播。每条 vdot 据此使用自己的配置快照，支持 vset 与向量操作交错执行。配置测试覆盖密集切换、合法与非法状态交替及取消后的恢复，逐层核对配置、源值和事务身份的配对关系。

旧目的寄存器不参与点积累计，译码需要避免无用旧目的依赖。目的可能是 v0，也

可能与某个源相同，因此源快照和目的写回类型必须明确。完成标记、异常信息和写使能与这一次微操作绑定，防止推测执行在取消后留下架构可见更新。

完整目的覆盖也使 H1 可以把结果寄存器收窄为 32 位。高 96 位对所有合法结果恒为零，可由输出端直接构造；ROB、pdest、异常和控制字段仍保存各自事务值。物理状态位数由实现结构决定，软件可见的 128 位结果宽度由指令合同决定，两者在写回接口处连接。

2.2.2 逐拍状态转移与握手

以下量在同一时钟边沿前计算，ready/valid 遵循 Decoupled 握手。P、R 各自调用该请求的 robIdx.needFlush(io.flush)，保留硬件 RobPtr 的 flag 与环绕语义。

killP = P.valid && P.rob.needFlush(flush)

killR = R.valid && R.rob.needFlush(flush)

liveP = P.valid && !killP && !reset

liveR = R.valid && !killR && !reset

out.valid = liveR

out.fire = out.valid && out.ready

rReady = !liveR || out.ready

move = liveP && rReady

pReady = !liveP || rReady

in.ready = pReady && parentAllowsVDot && !flush.valid && !reset

in.fire = in.valid && in.ready

nextR.valid = (liveR && !out.fire) || move

nextP.valid = (liveP && !move) || in.fire

move 为真时，R 锁存边沿前 P 的归约结果与身份；in.fire 为真时，P 锁存新输入的乘积与身份。两者同拍时分别使用旧 P 与新输入。reset 清除两个 valid 及父级占用，并抑制当拍握手；无效槽中的旧数据由有效位隔离。

表 2-4 逐拍状态转移与握手

边沿	接受	槽间转移	输出握手
E0	A 进入 P	无	无
E1	B 进入 P	A 进入 R	无
E2	C 进入 P	B 进入 R	A
E3	D 进入 P	C 进入 R	B

表中给出空管线、持续 ready 且无取消条件下的状态推导：E0 接收 A，E1 推进 A 并接收 B，E2 开始逐拍响应。相应测量在本章无阻塞父级事件图中展示。R 阻塞时由空 P 吸收一笔请求；两槽满后反压输入。部分取消保留未匹配事务，并允许幸存 P 进入已腾空 R。

重定向当拍抑制新请求接收，幸存旧 R 可握手，幸存 P 可推进。终结优先级为 reset、匹配取消、正常响应；同一事务只归入一个终结类别。reset 当拍输入、输出均无握手，复位丢弃数取沿前有效槽数；effectiveKillP=killP && !reset、effectiveKillR=killR && !reset 使取消计数与复位计数互斥。

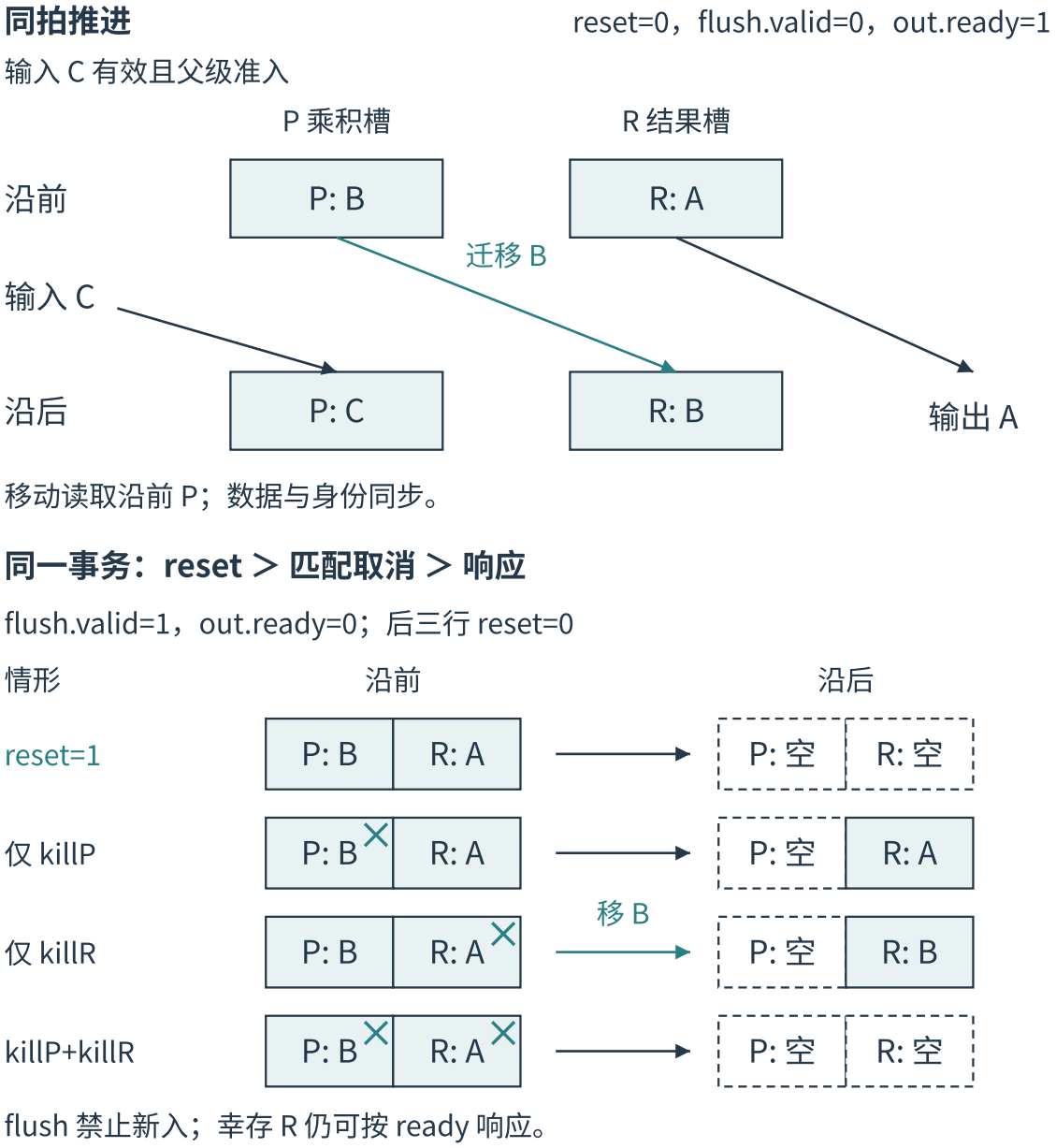


图 2-4　P/R 弹性状态与取消优先级

固定 0f 两槽行为：输入、旧 P 迁移和旧 R 响应可在同一边沿发生；move 读取沿前 P。P、R 各按自身 ROB 判断取消。reset 优先并抑制新入、迁移及响应；重定向当拍禁止新入，幸存 P 仍可填补空 R，幸存 R 仍可按 ready 响应。

```
    require(!cfg.writeFflags && !cfg.writeVxsat && !cfg.flushPipe && !cfg.replayInst)

    val pipelineStatus = IO(Output(new VDotElasticStatus))

>   private val pValid = RegInit(false.B)
    private val rValid = RegInit(false.B)
    private val pProducts = Reg(Vec(8, SInt(16.W)))
    private val pCtrl = Reg(chiselTypeOf(io.out.bits.ctrl))
    private val pPerf = Reg(chiselTypeOf(io.out.bits.perfDebugInfo))
    // H1 stores only the architectural low word; the public result stays 128 bits.
    // Keep each slot's complete control/perf identity on the same transfer edge.
    private val rPacket = Reg(new Bundle {
      val ctrl = chiselTypeOf(io.out.bits.ctrl)
      val res = new Bundle {
        val data = UInt(32.W)
      }
      val perfDebugInfo = chiselTypeOf(io.out.bits.perfDebugInfo)
    })

    pipelineStatus.pValid := pValid
    pipelineStatus.rValid := rValid
    pipelineStatus.occupancy := pValid.asUInt +& rValid.asUInt

    private val illegalInput = vill || vsew =/= VSew.e8 || vlmul =/= VLmul.m1 ||
      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
```

(a) 槽位寄存器与 32 位结果保存

```
PS C:\WINDOWS\system32> wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -Pattern 'private val rReady
=' -Context 5,12

      !vm || vstart =/= 0.U || vl =/= 8.U || fuOpType =/= VDotOpType.s8x8
    private val killP = pValid && pCtrl.robIdx.needFlush(io.flush)
    private val killR = rValid && rPacket.ctrl.robIdx.needFlush(io.flush)
    private val liveP = pValid && !killP && !reset.asBool
    private val liveR = rValid && !killR && !reset.asBool
>   private val rReady = !liveR || io.out.ready
    private val move = liveP && rReady
    private val pReady = !liveP || rReady

    // Parent admission is enforced by its atomic child dispatch. The FU also
    // suppresses every new accept on redirect/reset, including unrelated flush.
    io.in.ready := pReady && !io.flush.valid && !reset.asBool
    io.out.valid := liveR
    io.out.bits.ctrl := rPacket.ctrl
    io.out.bits.perfDebugInfo := rPacket.perfDebugInfo
    io.out.bits.res := 0.U.asTypeOf(io.out.bits.res)
    io.out.bits.res.data := Cat(0.U(96.W), rPacket.res.data)
```

(b) 握手与取消条件

```
PS C:\WINDOWS\system32>
                      > wsl -d Ubuntu-22.04 -- git -C /home/riscv/riscv-lab/zjhwork2-review-20261018/repo --no-pager show 0f12df6753785
56670afd0dd2153f47fcb330610:src/main/scala/xiangshan/backend/fu/wrapper/VDotElastic.scala | Select-String -SimpleMatch 'pValid := (live
P && !move) || io.in.fire' -Context 2,16

    acceptedCtrl.exceptionVec.get(ExceptionNO.illegalInstr) := illegalInput

>   pValid := (liveP && !move) || io.in.fire
    rValid := (liveR && !io.out.fire) || move

    when(io.in.fire) {
      pProducts := products.io.products
      pCtrl := acceptedCtrl
      pPerf := io.in.bits.perfDebugInfo
    }
    when(move) {
      // Reads pre-edge P even when a new request replaces it at this same edge.
      rPacket.ctrl := pCtrl
      rPacket.perfDebugInfo := pPerf
      rPacket.res.data := Mux(pCtrl.exceptionVec.get(ExceptionNO.illegalInstr),
        0.U(32.W), reduce.io.result(31, 0))
    }

    // Read-only handles for the separate G1/G2 observation harness. These carry
```

(c) 状态更新与旧 P 迁移

图 2-5 两槽寄存器与控制代码

依次展示寄存状态、握手与取消条件，以及读取旧 P 数据的状态更新。

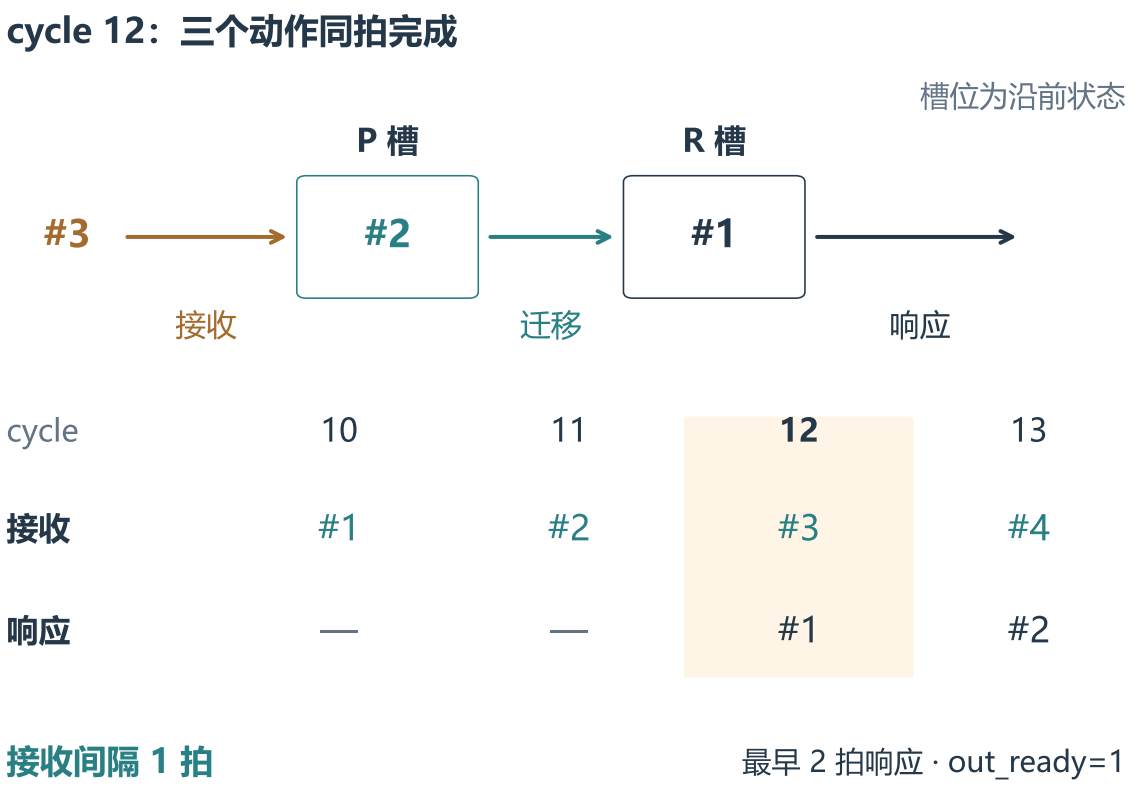


图 2-6　两槽在同一拍的接收、移动与输出

cycle 12 同时完成旧 R 响应、旧 P 迁移和新请求接收。槽位显示沿前状态，seq 表示测试事务号。

```
line 146 cycle=10984 O2_STATE
  pvalid=0 pseq=0 rvalid=0 rseq=0 occupancy=0 invalid=1 inready=1 outvalid=0 outready=1 move=0
line 148 cycle=10984 O2_ACCEPT
  seq=0 rob=1:31 pdest=110 vl=8 illegal=0 vecwen=1
line 149 cycle=10985 O2_STATE
  pvalid=1 pseq=0 rvalid=0 rseq=0 occupancy=1 invalid=1 inready=1 outvalid=0 outready=1 move=1
line 151 cycle=10985 O2_ACCEPT
  seq=1 rob=1:32 pdest=111 vl=8 illegal=0 vecwen=1
line 152 cycle=10985 O2_MOVE
  seq=0 rob=1:31 pdest=110 illegal=0 vecwen=1
line 153 cycle=10986 O2_STATE
  pvalid=1 pseq=1 rvalid=1 rseq=0 occupancy=2 invalid=1 inready=1 outvalid=1 outready=1 move=1
line 155 cycle=10986 O2_ACCEPT
  seq=2 rob=1:33 pdest=112 vl=8 illegal=0 vecwen=1
line 156 cycle=10986 O2_MOVE
  seq=1 rob=1:32 pdest=111 illegal=0 vecwen=1
line 157 cycle=10986 O2_RESPONSE
  seq=0 rob=1:31 pdest=110 data=0x00000000000000000000000000020000 vecwen=1

At cycle 10986: ACCEPT seq=2, MOVE seq=1, RESPONSE seq=0.
```

图 2-7　两槽重叠事务日志

clean-02 dense-batch128 日志中，10986 周期同时发生 ACCEPT seq2、MOVE seq1 与 RESPONSE seq0，分别对应新请求接收、旧 P 迁移和旧 R 响应。

2.2.3　VFEX4 派发与共享资源

VFEX4 包含 VDot、VIDiv 和 VFDivSqrt，共享输出采用 one-hot 选择和 ready 广播。两槽路径沿用互斥派发规则，确保任一时刻只有一个可完成来源：

1. 有任意 vdot 在途时，只能向 vdot 追加请求；DIV 禁止接收。

2. 任意 DIV 在途时，vdot 禁止接收，保持已有 DIV 串行行为。

3. vdot 最后响应或取消当拍不切换 DIV；在下一拍确认两个槽确实空、无旧 vdot 输出后，才可接受 DIV。

4. DIV 完成或取消后，确认旧输出撤销且子单元具备合法接收条件，再放行 vdot；切换根据实际有效状态与握手推进。

5. 跨 FU 切换保持输出有效来源唯一，DIV 的取消当拍行为按对应历史运行记录及全核合同核对。

6. 持续 vdot 供给可能推迟排空切换，沿用现有调度策略；混合负载记录等待，有限输入停止后检查排空与 DIV 最终完成。

O2 在 VFEX4 局部选择被选 FU 的 ready，并保持父子握手原子性。原 Dispatcher 的总 ready 由多个 ready 相与形成，child.valid 不与总 ready 同步相与，局部接入因此同时约束父级接收与子级接收。原有其他 EXU 派发语义保持不变，接口满足：

parent.in.fire == OR(all child.in.fire)

PopCount(all child.in.fire) <= 1

child.in.fire -> parent.in.fire && selected(child)

VDot 占用从两个真实槽状态取得，DIV 占用与身份独立保存。每次响应或取消只释放其对应事务，其他幸存槽继续有效。父级据此判断追加 VDot、等待排空或切换 DIV，保证双在途身份与资源占用一致。

输入选择依赖已注册占用、当前合法选择和下游 ready，valid/ready 路径保持无组合环。请求被选择但尚未握手时仍由上游保存，FU 在 in.fire 后才更新槽数据与身份。

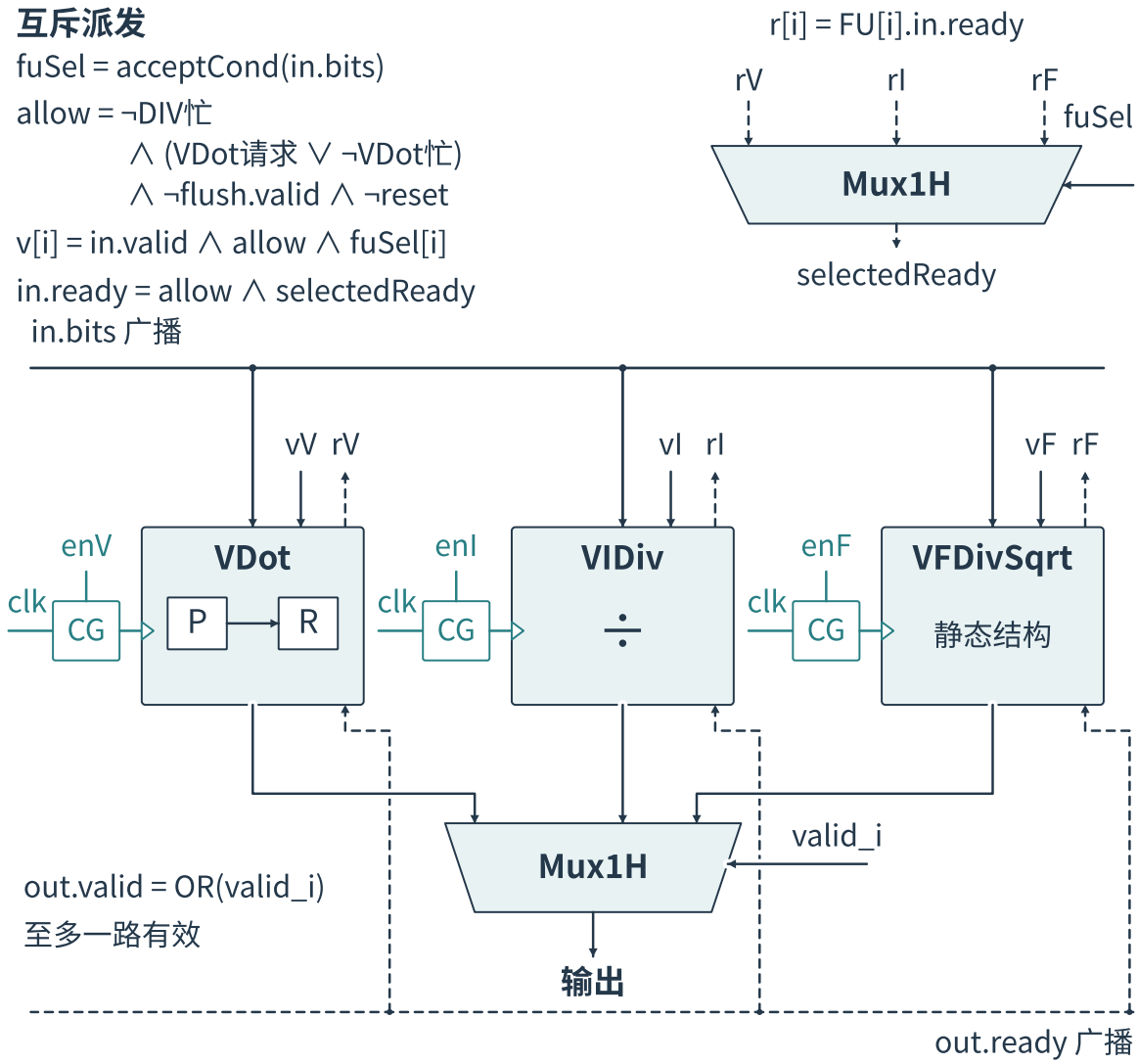


图 2-8　VFEX4 父级的派发、互斥与门控结构

固定 0f 且 VDotO2.enabled 与 EnableClockGate 开启时，VDot、VIDiv 和 VFDivSqrt 共享派发与写回资源，分别通过独立子时钟门控工作。VFDivSqrt 列仅表示静态结构，不能据此增加动态验证结论。

2.2.4　时钟门控、唤醒与写回

- 原单在途路径使用 uncer_en_reg，在 out.fire 且没有 in.fire 时清零。O2 可能仍有 P 事务，专用使能因而覆盖任一槽有效、当前输入接收以及必要复位和清理边沿。
- 当前槽有效时，即使 flush 将其全部取消，也必须保留处理本次清除所需的时钟边沿；排空后再关门。父级状态逻辑位于未门控时钟域。
- 原 O2 接入首先扩展 VDot 占用与门控，保留既有 DIV 路径；后续 0f 修复匹配 DIV 取消后的保持状态释放。真实父级验证使用 ClockGate，检查已关钟空槽复位、非空复位、cancel 与 reset 同拍以及重新唤醒，使复位传播和门控寄存器状态在实际时钟路径上对应。
- 完成通知、调度唤醒和写回仲裁沿用既有不定时延机制。输出尚未被接受时，R 持

续保持完整数据和控制包，等待真实写回 ready。

- 接入保持现有完成与唤醒关系，启动间隔通过实际父级输入握手测量；issue 反馈或写回竞争引起的等待记录在相应运行窗口中。
- 执行路径如需改为确定时延结构，需同时评审 FuConfig、控制流水、调度唤醒、寄存器读端口和写回预留；这些属于独立架构方案，采用单独合同。

2.3 基础功能验证

基础验证覆盖独立数学模型、算术 RTL、弹性 FU、真实 VFEX4 父级和全核差分五个层次。算术集合包含 22 组定向与 10000 组随机输入；全核矩阵包含 117 个不同用例，按原矩阵分片执行 124 次。0f 的 FU 与真实父级原记录及 4 + 16 项负控纳入复核，R5/G5 采用 13 项原记录与 1 项新控制诊断的组合验收。

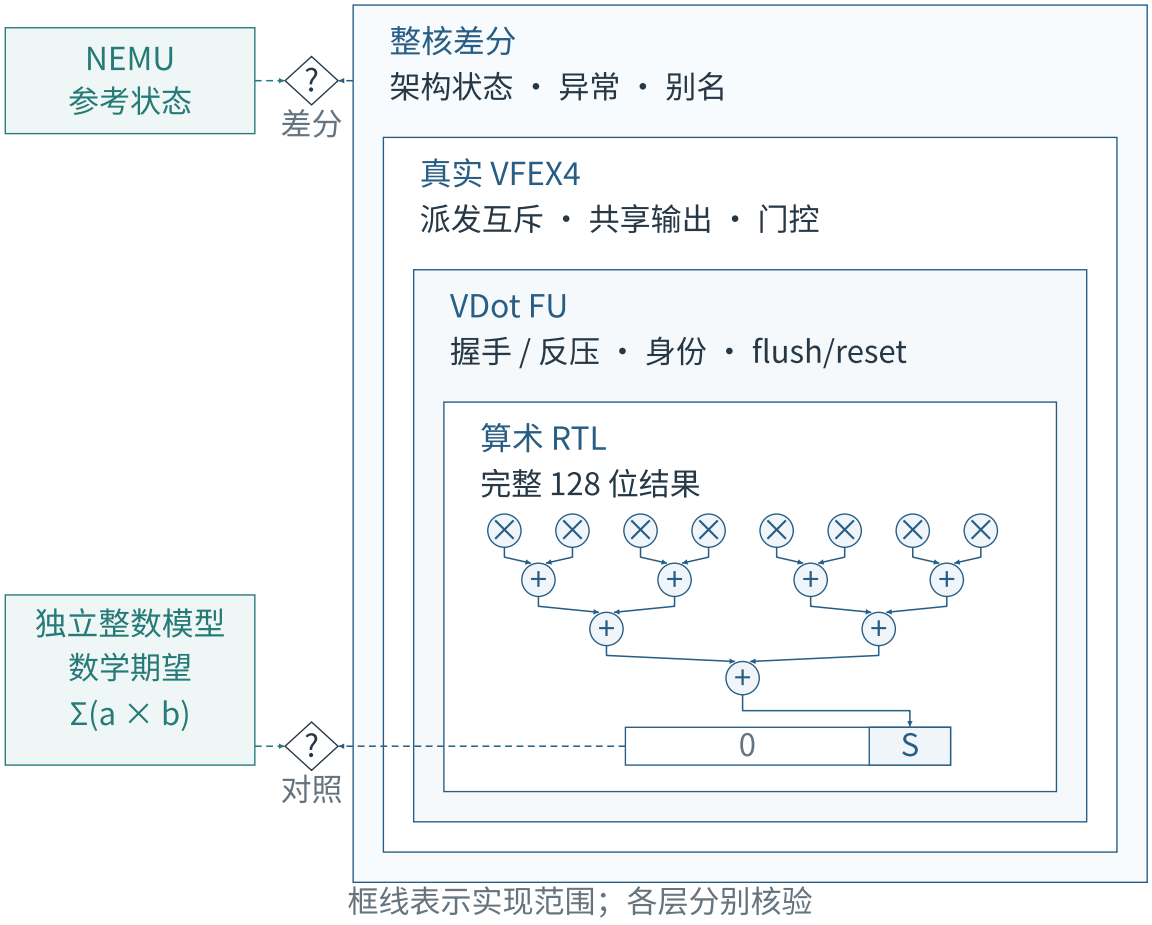


图 2-9 基础验证层级与覆盖范围

嵌套框表示 VDot 相关实现范围由算术 RTL 扩展到 FU、真实 VFEX4 父级和整核；各层分别核验并保留原证据。独立整数模型用于完整 128 位算术对照，NEMU 用于对应架构状态比较。实现包含不表示测试集合全包含，也不合并为一次新的 PASS。



图 2-10 主机参考模型测试结果

主机模型及测试数据检查共 35 个测试方法通过。

2.3.1 正数点积测试

正数用例将 [1,2,3,4,5,6,7,8] 与自身相乘并求和，精确结果为 204，目的低 32 位为 0x000000CC，高 96 位为零。该输入用于检查八项乘法、归约和完整向量写回；对应的正数事务响应与期望值在本节结果图中展示。

2.3.2 负数点积测试

负数用例计算 8 个 −1 与 8 个 1 的点积，精确结果为 −8，目的低 32 位为 0xFFFFFFF8，高 96 位为零。检查同时覆盖有符号输入解释、低 32 位补码以及完整结果格式；结果图保留对应负数事务的完整响应。

2.3.3 边界点积测试

八个 −128 与自身点积得到正端点 131072，目的低 32 位为 0x00020000；该值超过 18 位有符号数的正上限，用于检查归约末级位宽。八个 −128 与八个 127 点积得到负端点 −130048，低 32 位为 0xFFFE0400。零输入、端点及其他定向向量的精确预期集中列于下表；结果图展示正端点事务的完整响应。

```
VDOT_DATAPATH_SUITE_PASS suite=directed count=22
VDOT_DATAPATH_SUITE_PASS suite=random count=10000
VDOT_DATAPATH_PASS directed=22 random=10000 total=10022
```

(a) 组合算术 RTL 测试统计

```
[POSITIVE] functional-2 seq=1 ROB=0:39 pdest=103
vs1 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50807060504030201
expected sum      204
expected (128-bit) 0x000000000000000000000000000000cc
actual  (128-bit) 0x000000000000000000000000000000cc
ACCEPT cycle=339296 line=7030 | RESPONSE cycle=339298 line=7034
```

(b) 正数事务：期望值 204 与完整响应

```
[NEGATIVE] functional-0 seq=4 ROB=0:12 pdest=143
vs1 (128-bit)     0xa5a5a5a5a5a5a5a5ffffffffffffffff
vs2 (128-bit)     0xa5a5a5a5a5a5a5a50101010101010101
expected sum      -8
expected (128-bit) 0x00000000000000000000000ffffffff8
actual  (128-bit) 0x00000000000000000000000ffffffff8
ACCEPT cycle=420291 line=7927 | RESPONSE cycle=420293 line=7931
```

(c) 负数事务：期望值 -8 与完整响应

```
[BOUNDARY] functional-0 seq=5 ROB=0:43 pdest=66
vs1 (128-bit)     0xa5a5a5a5a5a5a5a58080808080808080
vs2 (128-bit)     0xa5a5a5a5a5a5a5a58080808080808080
expected sum      131072
expected (128-bit) 0x00000000000000000000000020000
actual  (128-bit) 0x00000000000000000000000020000
ACCEPT cycle=448417 line=8400 | RESPONSE cycle=448419 line=8404
```

(d) 边界事务：期望值 131072 与完整响应

图 2-11 正数、负数与边界用例的完整 128 位结果

组合 RTL 测试统计与三类全核观察器事务摘录。actual 取自响应记录，expected 按低 8 个 signed int8 输入计算。

2.3.4 完整定向向量与精确预期

[x]×8 表示重复 8 次。列表按 lane 0→7 排列，未另行说明时源高 64 位为零；输出高 96 位在所有用例中均为零。

表 2-5 完整定向向量与精确预期

ID	源 A 的低 8 项	源 B 的低 8 项	S（十进制）	目的低 32 位（十六进制）
D00	[0]×8	[127]×8	0	00000000
D01	[1]×8	[1]×8	8	00000008

表 2-5 完整定向向量与精确预期（续）

ID	源 A 的低 8 项	源 B 的低 8 项	S（十进制）	目的低 32 位（十六进制）
D02	[1,2,3,4,5,6,7,8]	同 A	204	000000CC
D03	[-1]×8	[1]×8	-8	FFFFFFF8
D04	[-1]×8	[-1]×8	8	00000008
D05	[127]×8	[127]×8	129032	0001F808
D06	[-128]×8	[127]×8	-130048	FFFE0400
D07	[-128]×8	[-128]×8	131072	00020000
D08	[1,-2,3,-4,5,-6,7,-8]	[8,7,6,5,4,3,2,1]	0	00000000
D09	[-128,0,0,0,0,0,0,0]	[-1,0,0,0,0,0,0,0]	128	00000080
D10	[0,0,0,0,0,0,0,-128]	[0,0,0,0,0,0,0,127]	-16256	FFFFC080

2.3.5 数据布局、别名与异常测试

表 2-6 数据布局、别名与异常测试

ID	测试内容	判定标准
L01	逐 lane 一热：只令 A 的 lane i 非零，B 的 8 项互不相同，i 遍历 0～7	每次等于对应那一项乘积，检测遗漏和单侧错位
L02	保持低 8 项不变，两个源高 64 位分别换成随机值、全 FF、交替 55/AA	结果完全相同
L03	独立目的寄存器预填全 FF，再执行 D03	目的完整值为 00000000_00000000_00000000_FFFFFFF8
L04	不重叠、vd=vs1、vd=vs2、vs1=vs2、三者相同；含 v0/v31	使用实际执行前源快照算期望；别名不能使源被提前破坏
L05	四种 vta/vma 组合	每种均得到相同完整目的值，保留原配置

表 2-6 数据布局、别名与异常测试（续）

ID	测试内容	判定标准
S01	vxrm 四种值、vxsat 0/1 组合，预置后执行合法指令	算术结果不变；上述状态及其 vcsr 映射保留
S02	检查 vl/vtype/vstart、非目的寄存器与内存	除正常提交、向量置脏等既有机制外，无额外副作用
E01	逐项改变 SEW、LMUL、vl、vstart、vill，或关闭向量状态	模型显式拒绝；完整指令实现发生非法指令异常，无部分结果
E02	vm=0、逐个翻转固定 funct 位；合法寄存器字段遍历	只识别精确 mask/match；确认相邻模式未被误译码为 vdot
C01	连续请求、输出反压、结果依赖、同一目的连续写、flush	无丢失/重复/错序提交，取消后无已提交写回
R01	原 hello/vadd 及相关向量回归	保持正确；vdot 新路径不破坏旧指令

L01 针对遗漏 lane 与单侧错位。两源同时采用相同排列时数学点积保持不变，这一情况归入模型不变量检查；源字段位置与寄存器依赖由译码、重命名观察点核对。CSR 及非法配置用例使用合法准备指令，保存目标 vdot 执行前的实际 CSR 状态。

非法配置通过架构可达状态构造。标准 vset 指令设置非法 vtype 时，vill=1 与 vl=0 联动，记录按这一联合状态核对目标 vdot 的非法异常及无部分更新。非零 vstart 在向量配置和加载完成后写入；向量关闭用例先准备数据，再关闭 VS，保证被测异常来自目标指令。

2.3.6 随机输入与完整结果读回

- 初始数学模型和算术计划采用固定种子 0x20261011 及至少 10000 组 signed int8 输入，全核冒烟使用定向向量和 32 组固定随机输入。该计划记录与本章 22+10000 算术集合、117 不同全核用例及 124 次执行的实际统计分别标识。
- 保存生成器版本、实际输入、完整期望值和合同版本，种子用于重新生成同一算法下的序列；跨语言复现直接对照归档输入。
- 交叉检查交换两个源、同时重排两边低 8 项等不变量。涉及 int8 取负时排除 -128，避免把不可表示的 +128 当成合法输入。
- 失败应输出指令字、PC、配置、源/目的编号、指令前源值、完整实际/期望目的值；定向用例输出用例 ID。

- 独立模型、算术 RTL、FU/父级、NEMU 和全核 DiffTest 分别记录观察对象与结果，使用同一数学定义连接各层检查。

软件读取结果覆盖全部 16 字节。例如指令执行后切换到 e32,m1,vl=4，保存四个 32 位元素，再恢复约定的 vl/vtype。读回程序记录准备、结果保存和状态恢复三个阶段；被测指令的副作用检查以执行前后指定观察点比较。

2.4 协议与系统验证

2.4.1 反压期间的结果与身份保持

io.out.valid 表示 R 中存在可响应事务，io.out.ready 决定该拍能否完成握手。反压期间，R 槽的结果、ROB 标识、目的物理寄存器、异常向量、写使能和性能字段作为一个整体保持。检查器逐字段核对这个输出包，使数值与写回目标同时受约束。

验证为每个接受事务记录独立身份，响应时与该身份的参考值对应。负控包括交换两事务 pdest、改变 ROB、篡改高位、制造重复响应或漏响应。实际硬件协议测试与检查器日志变异分别记账，前者验证设计，后者验证证据审查能力。

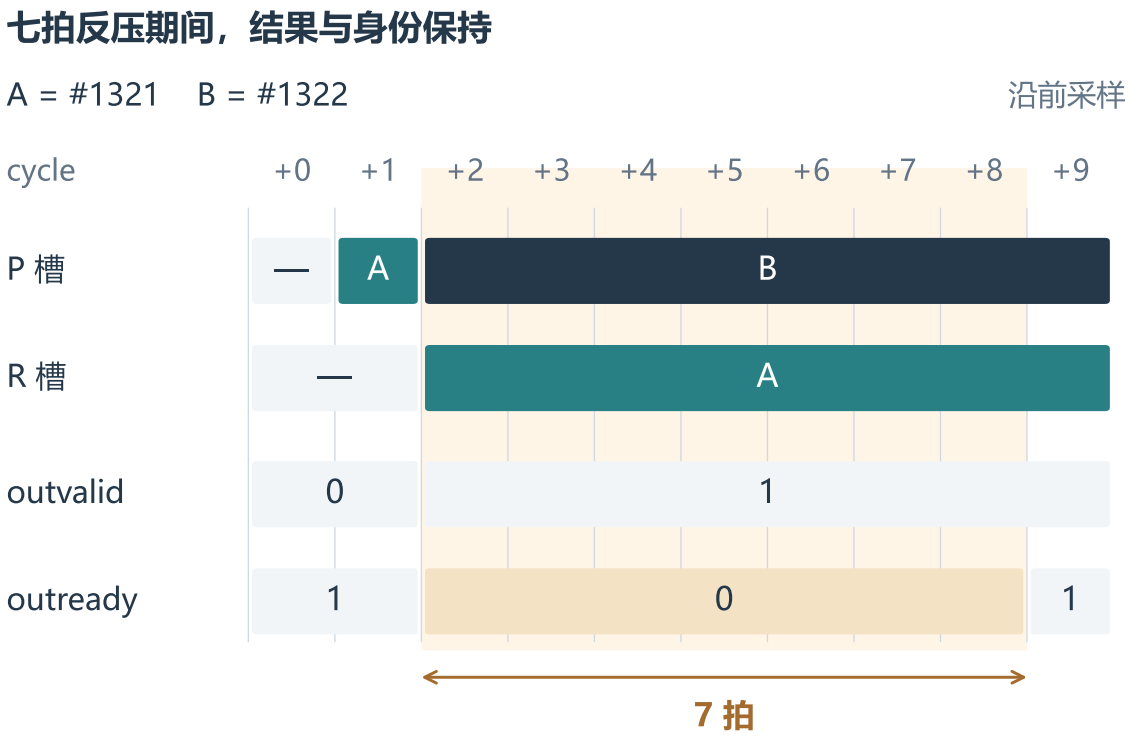


图 2-12 反压期间的结果与事务保持

seq1321 在 181599—181605 连续反压 7 拍，完整结果为 0x0000000000000000000000000001A87。181606 响应一次，同时旧 P 事务 1322 转入 R；FU vready 恢复，父级 inready 仍为 0。

2.4.2 ROB 取消与复位优先级

redirect 按 RobPtr 的 index 与环绕 flag 共同判断 P、R 的程序年龄。测试覆盖 flag 变化、flushItself 两种取值以及接受顺序与年龄顺序不同的情况。每个窗口先确定应保留和应取消的身份集合，再核对幸存事务输出与被取消槽的有效位清除。

同一拍发生输出 ready 和匹配取消时，取消决定该事务是否仍可形成响应。reset 则清除槽有效状态，并检查接口握手被抑制。真实父级还需验证已经关钟的空槽可以正确复位，以及取消后重新唤醒的第一笔请求不丢失。

任意前缀都满足事务守恒。接受数等于已响应数、已取消数、复位丢弃数和当前在途数之和。当前在途数最大为 2。这个关系能够发现尾部漏计、幽灵响应和被取消事务再次输出等错误，并要求正常结束时排空。

例如沿前两槽有效且 reset 与匹配 redirect 同拍时，优先级使两项计入复位丢弃，取消和响应不再重复计数。仅 R 取消而 P 幸存时，取消集合增加一项，P 转入 R 保持原身份，在途数随之减一；槽间移动本身不增加接受数。守恒式把这些不同动作统一到同一个数量关系中。

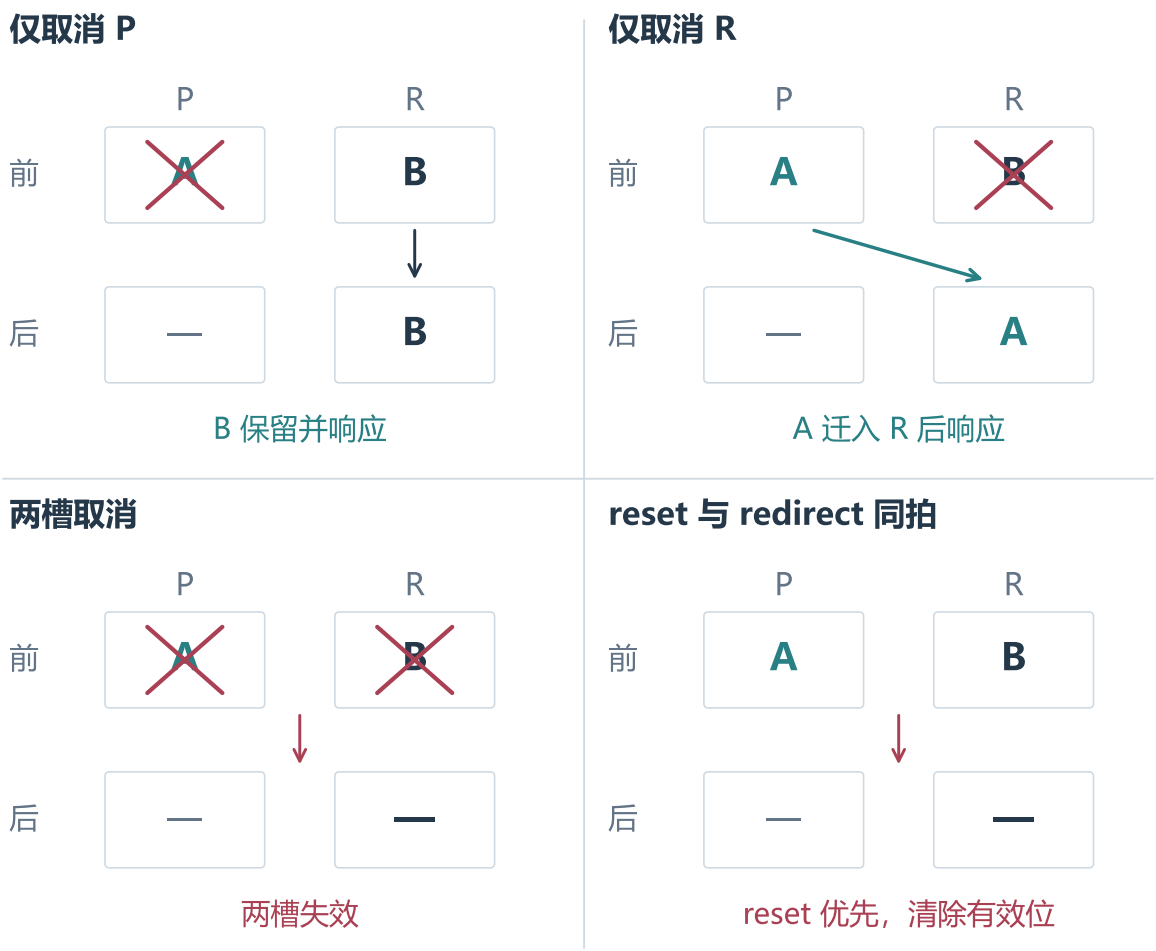


图 2-13　取消与复位的槽位变化

四个独立场景分别展示仅 P 取消、仅 R 取消、两槽取消和 reset 优先清除。A/B 为各场景内的事务标记，取消依据 ROB 年龄。

2.4.3　DIV 切换、取消清理与关钟

VFEX4 负责 VDot 与 DIV 的互斥派发、唯一完成源选择和占用释放。动态验证使用真实 ClockGate，覆盖 VDot、整数 DIV、浮点 DIV 之间的切换，以及反压、取消、复位和重新唤醒；SQRT 取消路径采用静态检查。图中以时钟上升沿后的子时钟采样状态展示取消清理和后续空闲关钟。

资源切换可按占用、派发和完成三个观察点理解。占用说明旧操作是否仍在途，派发说明新操作是否真正握手，完成说明共享输出当前来自哪一类 FU。切换前核对旧输出撤销与子单元接收条件，切换后核对首次接收与响应身份，能够把 busy 释放问题与结果选择问题分开定位。

0f 修复匹配 DIV 取消之后保持状态的释放：取消当拍保留完成清理所需的时钟边沿，随后解除旧占用；仍在途的有效事务继续获得时钟。测试同时观察空闲采样与幸存事务完成，核对取消、资源释放和新请求进入之间的顺序。



图 2-14　DIV 切换、取消清理与关钟

VDot 排空后接收整数或浮点 DIV，取消清理完成后子时钟归零，再接收新 VDot。图中 clocks 为沿后采样的子时钟位掩码。

2.4.4　写回身份与全核结果检查

向量点积的数学参考值可以由两个源向量独立计算，但它不能告诉检查器该值是否写入了正确目的。测试平台因此为每个接受事务绑定一个身份记录，包含目的物理寄存器、ROB 位置、指令控制及期望结果。响应到达时先核对身份，再核对完整一百二十八位数据。若先按结果值寻找匹配事务，两笔恰好得到同一结果的请求可能被错误互换而不被发现。零结果、相同源向量和不同目的寄存器的交错请求尤其适合暴露这一盲点。

写回使能与异常字段同样需要随身份核查。异常指令可以在执行路径中占有状态，但不能以正常结果覆盖架构目的；被取消的指令在局部槽位里保留旧数据，也不能因 ready 恢复而重新输出。检查器在每次接受、响应和取消时更新账本，并在结束时核对所有身份的去向。该账本提供了比单纯比较输出顺序更强的约束，也能分辨协议漏响应与结果错误。

完整路径复验还应覆盖目的别名和连续配置变化。若两条指令的源与目的存在交叉别名，重命名阶段必须固定各自的源版本；若配置指令夹在两条点积之间，执行端必须持有各自对应的 VL 与 vtype 快照。最终软件结果可能在少数输入上偶然相同，因而需要结合调度记录、响应身份和异常状态判断。只有这些字段与数学结果共同成立，才可将局部算术通过提升为全核写回路径通过。

2.4.5 控制场景的组合与回归覆盖

回归矩阵按失效机理组织：极值与随机输入覆盖算术，配置交错覆盖依赖，别名覆盖源快照，反压覆盖包稳定，取消覆盖年龄和复位优先级，混合操作覆盖共享资源。每项记录主断言及额外观察点，失败时据此定位输入和轨迹，实现变化时据此选择受影响的回归范围。

为了避免组合爆炸，可先对每一维构造最小可区分情境，再挑选交叉点。比如正常单事务可检查基础算术和响应格式；双事务且不同目的寄存器可检查连续接受；双事务加反压可检查载荷稳定；环回 ROB 加选择性取消可检查年龄语义；取消后首笔 DIV 和点积交替可检查父级释放。并不是所有输入值都要与所有控制状态笛卡尔相乘，但每个高风险交叉都应能在矩阵中定位到明确的用例和预期。

随机测试以种子、生成规则和实际输入共同确定可重放序列，并覆盖正负数、零、高位哨兵及别名组合。精确的 18 位溢出正端点由定向用例构造，随机输入补充一般数据分布。两类记录在同一数学定义下分别提供可解释的边界检查和较宽的数据覆盖。

2.5 设计演进、参数扩展与取舍

O2 通过两槽容量支持连续接收，H1 降低结果寄存器开销，0f 完善 DIV 取消后的关钟行为。冻结 0f 构建按 clean-02 配置启用 O2/T14、关闭 O1；通用源码默认开关与该冻结构建分别记录。一拍 P4 候选完成局部验证后进入 47 项同约束筛选，初版 6 项、唯一修订 10 项未通过，因此生产版本选择 0f。

参数扩展实验实现 signed INT16×8 和 64 位结果，完成 94 组定向及 10000 组随机验证，用于检查算术与包装层的参数化方法。当前生产 ISA 仍采用 INT8×8、低 32 位结果及高 96 位清零的合同。

指令合同 • 稳定 FU 源码 • R5 验收矩阵 • 参数扩展

2.5.1 吞吐与实现成本

弹性槽提供的是输入与输出之间的缓冲容量，它改善持续请求或短暂下游阻塞时的利用率，却引入额外的状态寄存器和 ready 路径。对独立指令流，流水重叠可以降低平均每条指令的间隔；对严格依赖链，下一条必须等前一条写回，缓冲容量不一定产生同样收益。独立流与依赖链分别测量，使用各自的周期结果解释流水收益。

综合成本也有层级。乘法树位宽、P/R 寄存器规模、控制比较器以及父级选择逻辑分别贡献不同路径和面积。H1 收窄结果状态可能减少寄存器位数，但若增加末端拼接或改变布线，最差时序端点可能转移。P4 缩短内部算术拍数也可能使组合路径穿越更多逻辑，导致父级 ready 或门控使能的余量变差。综合记录同时列出路径端点、约束、面积和裕量，解释各项成本变化。

2.5.2 本阶段小结与接口衔接

阶段间接口以架构语义冻结：参考模型接收指令字、源向量、控制状态和异常条件，返回完整目的向量及异常结果。数学和指令模型独立于 P/R 具体拍数，允许硬件在保持架构效果的前提下调整内部流水。性能记录则绑定实际硬件版本和开关，描述该实现的周期与成本。

软件输入镜像、编译器与汇编路径、配置前置条件一同冻结。C 接口或链接布局变化后，检查最终指令字与调用、返回约定，分别记录裸指令部分和软件包装部分的开销。差分验证沿用数学与异常合同，性能测量明确入口、出口和状态恢复要求，两类记录通过同一指令语义衔接。

2.6 复现资料与接口索引

2.6.1 汇编封装与编码冲突记录

软件通过独立.S 文件中的.word 0x0284850b 或校验后的编码宏生成机器字，宏同时检查寄存器编号范围。反汇编工具可显示机器字或识别后的名称；指令身份以机器字、字段和最终链接映像为准。C 接口的调用和返回约定见第三阶段软件集成说明。

C 内联汇编需要明确寄存器绑定、clobber 和编译器重排约束，使编译器可见的读写集合与实际向量操作一致。独立汇编封装则通过显式入口、出口与保存恢复约定表达这些关系。两种路径都检查最终目标指令和结果读回。

2026-10-11 在固定 XiangShan 实际译码表和 NEMU 主译码表进行的静态检查未发现与 vdot mask/match 交叠的已接入模式。已有 nemu_trap/TRAP 使用 0x6b。rocket-chip 依赖中 CUSTOM0 通配项属于 Rocket RoCC 译码体系，当时未接入香山实际译码表；启用该路径时需重新核对分配与仲裁。这份记录说明接入前的编码空间，后续译码和全核结果由本章功能验证给出。

2.6.2 源码接入位置与职责

表 2-7 源码接入位置与职责

子系统	相对路径	接入职责
XiangShan 指令模式	src/main/scala/xiangshan/backend/decode/Instructions.scala	精确 vdot 模式、相邻编码及既有扩展的模式关系
XiangShan 向量分类	src/main/scala/xiangshan/backend/decode/isa/bitfield/RiscvInst.scala	将精确 vdot 模式纳入向量分类；同时保留其他 custom 编码的独立分派
XiangShan 译码控制	src/main/scala/xiangshan/backend/decode/DecodeUnit.scala	正确的向量源/目的类型、配置依赖、合法性检查、微操作和写回控制
XiangShan 执行接入	src/main/scala/xiangshan/backend/fu/FuConfig.scala，fu/wrapper/，fu/vector/	执行资源、结果位宽、完整目的写回、取消和不定时延接口
NEMU 主译码	src/isa/riscv64/instr/decode.c	custom 主 opcode 分派、向量状态检查及独立 vdot 执行入口
NEMU 向量实现	src/isa/riscv64/instr/rvv/decode.h、exec.h、vcompute.h、vcompute_impl.c	字段解析、配置检查、精确算术及本指令的完整目的写回规则
NEMU 寄存器访问	src/isa/riscv64/instr/rvv/vreg.h、vreg_impl.c	有符号源读取、先读后写、完整 128 位写入
NEMU 声明与差分状态	src/isa/riscv64/include/isa-all-instr.h、src/isa/riscv64/difftest/ref.c	执行入口声明、向量和 CSR 状态同步，确认实际比较范围

表中路径用于定位接入职责，源代码行号对应当时固定的硬件和 NEMU 提交。参考入口：XiangShan 译码目录、NEMU 主译码、NEMU RVV 目录。

固定版本源码显示以下接入关系：

- RiscvInst.scala:93 的向量分类需包含精确 vdot 模式；DecodeUnit.scala:897 的向量禁用检查同时依赖 FU 分类。编码匹配与功能分类分别完成后，合法性检查才能覆盖该指令。
- DecodeUnit.scala:775 的输入带 vtype/vstart，未直接带实际 vl；vl=8 限制须通过正确的 VL 依赖，在能取得 srcVConfig.vl 的路径检查，例如 fu/vector/VecPipedFuncUnit.scala:54，并接通非法指令异常输出。
- DecodeStage.scala:236 对 v0 目的使用独立写使能，重命名也有相应通路；寄存器测试因此包含 v0 及 v31，并核对目的物理映射。

- 原 NEMU 的 instr/rvv/decode.h:167 在 vopivv 子表中将 funct6=0 解释为 vadd，不再检查主 opcode。vdot 使用独立执行分派，复用字段解析和寄存器访问；向量状态检查与置脏沿用 instr/vp.c 的 vp_enable() 和 vp_set_dirty()。

2.6.3 参考模型的构建与运行身份

独立数学模型按精确整数定义计算参考值，NEMU 在其外层实现编码、配置、异常和架构状态更新。差分比较包含完整目的结果及约定状态；可控故障参考与正常参考恢复分别检查错误检出和正常运行。参考模型与 DUT 使用独立的数据计算路径。

共享参考库与 standalone NEMU 分别构建，记录各自提交、配置、产物摘要和实际加载路径。团队维护可访问的源码版本或可复现补丁，运行记录绑定所加载的具体库，保证其他工作区能够还原同一指令模型。

2.6.4 事件记录与检查不变量

O2_SCHEMA version=1 按未门控全局 cycle 记录两槽事件。每次父级接收产生唯一观察 seq，P 至 R 转移保持同一 seq，检查器以此串联接收、等待、取消与响应；seq 只服务于观察，功能决策使用真实槽状态和 ROB 身份。

事件集合包含父级接收与等待、请求种类及选择、ready 及阻塞原因、P_ACCEPT、P_TO_R、R_STALL、R_RESPONSE、P_KILL、R_KILL、RESET 丢弃、槽占用、DIV 开始结束和重定向。检查按统一 cycle 处理旧事务终结、移动和新接收，同时保留原始输出顺序以便定位日志。

必须满足：

- 任意前缀：接受数 = 响应数 + 取消数 + reset 丢弃数 + 当前在途数；同一事务恰好属于一个终结类别。
- 当前在途数在 0..2；两个槽不能持有同一 seq；输出来自 R 中那个身份。
- 幸存输出按接受顺序，不允许重复/幽灵响应；完整 128 位及异常/写使能与独立模型一致。
- 多槽取消逐笔按 ROB 年龄判断，覆盖年龄顺序与接受顺序相反、flag 环绕、flushItself 两种情况。
- 反压保持整个 R 包稳定；只有匹配取消/reset 可以撤销旧响应。
- 父子握手一致，父级 out 与被选子输出一致，FU 输出 one-hot，门控不能冻结未终结请求。
- 正常运行结束时所有已接收事务均有终结记录；主动故障终止保留已检查前缀与停止位置，单独记录运行状态。

2.6.5 原 O2 分阶段判据与负控

下表保留原 O2 从设计到采用的分阶段判据，说明各类验证之间的衔接。当前 0f 的完成情况由本章功能验证汇总及 R5 验收矩阵记录；动态父级记录覆盖 VDot、整数 DIV 和浮点 DIV，SQRT 保留静态取消路径检查的状态。

表 2-8 原 O2 分阶段判据与负控

门禁	核验内容	判据
G0 评估/合同	旧轨迹复算、两条接入路线审查、候选范围及停止条件	形成隔离原型的输入合同与停止条件
G1 算术与弹性 FU	原 22+10000 完整 128 位向量；连续满管线、两槽反压/腾空；合法非法交错、v0 交错；部分/全部取消、reset 和环绕	无反压时真实 II=1 且最早响应 2 拍；事务守恒与反例检出通过
G2 真实父级	真 ExeUnit+VDot+VIDiv+VFDivSqrt+ClockGate；全部 FU 切换、满管线反压、cancel 与切换重合、持续供给及最终排空	无误派发/双完成/误关钟；其他 FU 受影响范围回归通过
G3 全核	新 O2 EMU、正常 NEMU；hello/vadd、117 唯一功能/异常矩阵、T12 等价控制压力；故障参考库及正常恢复	全部 DiffTest/完整状态通过，新观察事件覆盖达标
G4 公平性能	冻结相同 16 镜像、参考库、工具配置，O1/O2 各自实测 custom 与 RVV；同合同四场景和相同样本规则	有实际净收益、退化可解释、结果与身份绑定可复算
G5 采用与交付	O3 应用数据、O4 结构/时序证据、O5/T17 独立干净检出构建及相应复核记录	形成最终模式、冻结版本与交付记录

定向负控覆盖 P/R 身份交换、结果高位篡改、漏取消或误杀幸存事务、重复或丢失响应、反压字段变化、父子握手不一致、双 FU 有效及在途误关钟。硬件注入与日志/检查器注入分别记录故障位置、预期断言和实际检出，保持各自的观察层级。

原 O2 成本筛选将冻结 batch128 相对 O1 降低至少 10% 设为工程目标，即由 362.50 降至不高于 326.25 cycles，同时检查其他保留场景是否出现超过 2% 的均值退化。若 RVV 为 678，该目标约对应 2.078×；正式加速比使用同一候选自身实测 RVV。该目标属于当

时的性能筛选条件，采用判断还结合功能、协议、资源、时序和复现结果。

2.6.6 RVV 对照的功能等价条件

RVV 对照以 8 位有符号输入、16 位精确乘积和 32 位扩宽归约完成相同功能。序列围绕 vwmul.vv 与 vwredsum.vs 组织，中间 SEW/LMUL、寄存器分组、零初值和最终高位清零共同构成功能等价条件。具体冻结序列与计时结果见第三阶段“RVV 对照与周期测量方法”。

基线按以下条件核对：

1. 只计算源低 8 项，从零开始，不把旧目的或源高半部分加入。

2. 不使用会在 8/16 位提前截断的组合；D06、D07 必须通过。

3. 最终目的低 32 位及高 96 位与“点积定义、位宽与结果布局”小节完全相同；普通归约指令的尾部策略不能代替显式验证。

4. 双方在共同入口、出口比较完整 128 位目的结果及约定向量 CSR，vl/vtype 恢复实际入态。临时 scratch 寄存器预先列明并排除于该比较集合，源目的别名用例单独核对源快照。单条 vdot 的严格副作用规则与多指令序列的临时寄存器约定分别定义。

5. 分别记录核心算术窗口和完整等价序列的周期；清零、配置切换、恢复及搬运按共同边界计入，两边的装载、存储计时口径一致。

6. 使用同一硬件基线、输入和测量边界，分别说明依赖链延迟与独立流吞吐。性能指标采用 DUT 周期，主机运行时间用于执行资源记录。

冻结测量记录保存具体序列、入口出口、计时事件和样本规则，详见第三阶段“RVV 对照与周期测量方法”。设计语义确定功能比较集合，测量约定确定周期比较范围，两者共同连接裸指令测试与软件调用实验。

2.6.7 待确认规格与变更记录

下表集中列出内部 v0.1 合同涉及的赛事待确认问题。现有归档尚无命题方正式答复，当前实现和测试依表中内部约定组织；官方口径如有更新，再按变更流程调整对应合同。

表 2-9 待确认规格与变更记录

编号/优先级	需要确认的问题	本版约定及受影响内容
Q01 / P0	图中的 8 项是否为最终验收语义？是否明确取低编号 8 项？	固定低 8 项；影响读取范围、算术规模、测试和基线

表 2-9 待确认规格与变更记录（续）

编号/优先级	需要确认的问题	本版约定及受影响内容
Q02 / P0	“vd 第一个元素”的位宽是什么？是否接受低 32 位精确结果、高 96 位清零？	“点积定义、位宽与结果布局”小节；影响目的格式、写回和完整结果比较
Q03 / P0	是否要求精确结果？是否另有截断、饱和或旧 vd 累加规则？	精确从零求和；影响数据通路与边界结果
Q04 / P0	vl/LMUL/mask/vstart 的必测范围是什么？是否接受“指令编码与合法配置”小节的受限原型域？	固定 e8/m1/vl8/vstart0、无掩码；影响控制及异常范围
Q05 / P1	vta/vma 是否影响“其他通道清零”？是否要求源目的重叠支持？	策略位不改变完整结果，全部单寄存器别名支持
Q06 / P1	是否有指定或禁止的指令编码？是否允许使用 custom-0？	“指令编码与合法配置”小节；影响 RTL、NEMU 与汇编封装同步
Q07 / P1	延迟比较是否有指定输入/出口状态、计时边界或基线要求？	按“RVV 对照的功能等价条件”小节先验证等价，再分别报告各项开销

正式答复按日期、来源及原文归档到 team/spec/clarifications，并与 Q01～Q07 逐项关联。记录同时保留现有约定、答复后的约定和受影响测试，便于重新核对实现与评分口径。

行为变更记录旧行为、新行为、依据和受影响用例，经评审后升级合同标识，同步更新 RTL、NEMU、编码封装与数据。原版本输入和结果继续保留原标识，新版本使用自己的验证与性能记录。

2.6.8 技术资料索引

内部 ISA 合同、数学边界和测试向量；两槽弹性状态、父级协议及验证不变量。